

Some Practice Problems (ADA 2023, CSE 222)

Diptapriyo Majumdar

April 24, 2023

Divide and Conquer

Recurrences

Dynamic Programming

Greedy Methods - Huffman Coding,

Graph Traversals - BFS, DFS and their applications.

Dynamic Programming on directed acyclic graphs.

Kruskal's Algorithm - spanning tree

Shortest Paths - Dijkstra's Algo

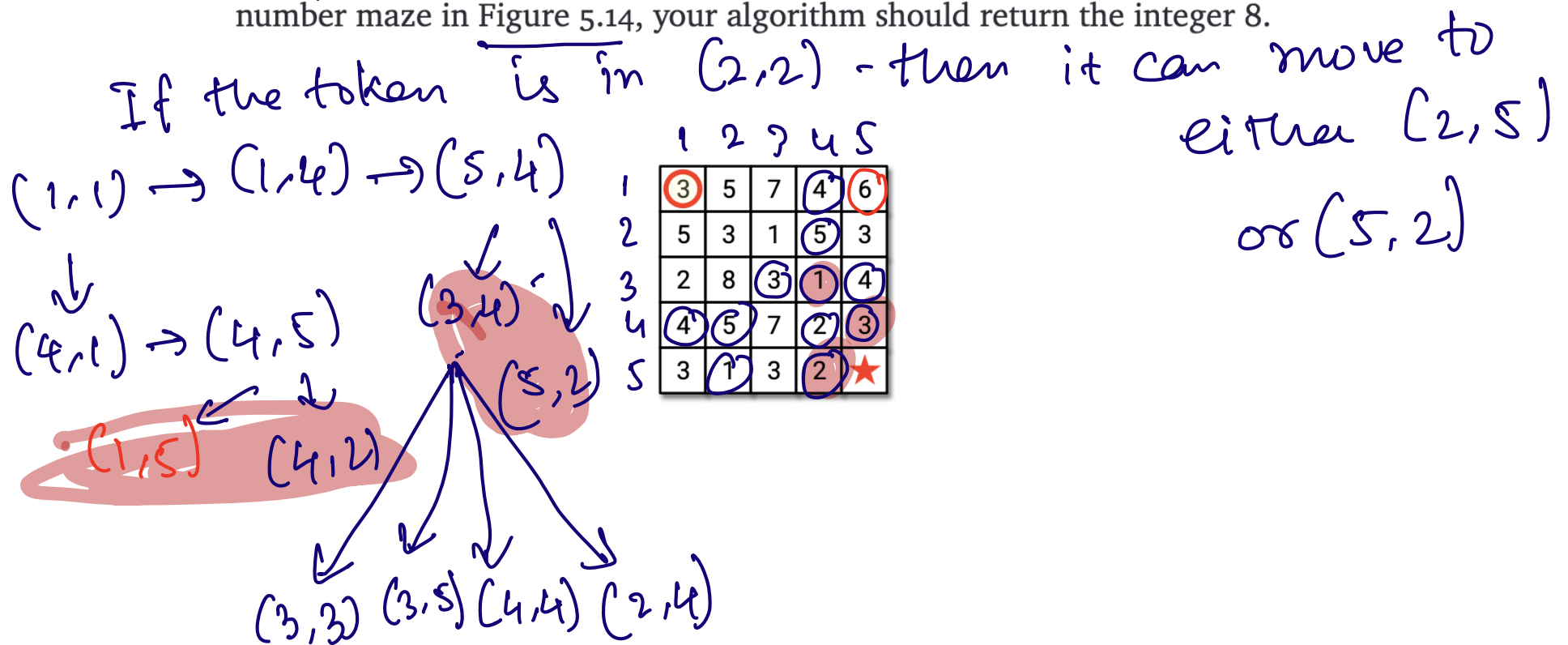
Bellman Ford

Network Flow, Max-Flow Min-cut Theorem.

NP-Completeness and Polynomial-time reductions.

A **number maze** is an $n \times n$ grid of positive integers. A token starts in the upper left corner; your goal is to move the token to the lower-right corner. On each turn, you are allowed to move the token up, down, left, or right; the distance you may move the token is determined by the number on its current square. For example, if the token is on a square labeled 3, then you may move the token three steps up, three steps down, three steps left, or three steps right. However, you are never allowed to move the token off the edge of the board.

Describe and analyze an efficient algorithm that either returns the minimum number of moves required to solve a given number maze, or correctly reports that the maze has no solution. For example, given the number maze in Figure 5.14, your algorithm should return the integer 8.



Graph Construction: Directed graph

Set of vertices: $V(G) = \{\text{indices of the grid}\}$
 $= \{(1,1), (1,2), (1,3), \dots, (n,n)\}$

Label of a vertex: $\text{Label}(i,j) = \text{number written at the grid location } (i,j)$
 $1 \leq a_i, a_j, b_i, b_j \leq n$

Set of edges: A pair of indices (a_i, b_i) and (a_j, b_j) . There is an edge $(a_i, b_i) \rightarrow (a_j, b_j)$

if (i) $\text{Label}(a_i, b_i) + a_i = a_j$
and $b_i = b_j$

OR

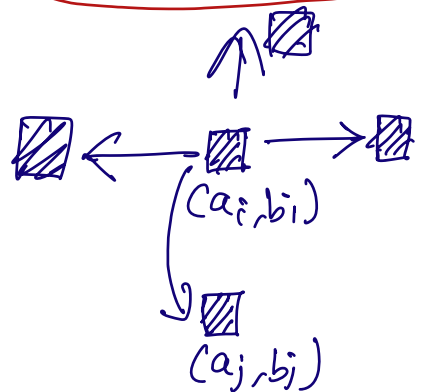
(ii) $\text{Label}(a_i, b_i) + b_i = b_j$
and $a_i = a_j$

OR

(iii) $a_i - \text{Label}(a_i, b_i) = a_j$
and $b_i = b_j$

OR

(iv) $b_i - \text{Label}(a_i, b_i) = b_j$
and $a_i = a_j$



weight of every directed edge is 1.

Algorithm: Invoke Dijkstra's Algorithm with source vertex $(1,1)$, i.e. the top-left corner.

After Dijkstra's Algorithm terminates, check whether the distance $[(n,n)] = \infty$ or not. If $\text{dist}[(n,n)] = \infty$, then output that no solution exists.

Else, $\text{dist}[(n,n)]$ is finite, then output distance $[(n,n)]$ value.

Correctness: follows from the correctness of Dijkstra's Algorithm with non negative weighted edges.

Running Time:

What is the number of edges of G ?

Every vertex has at most 4 neighbors with outgoing edge

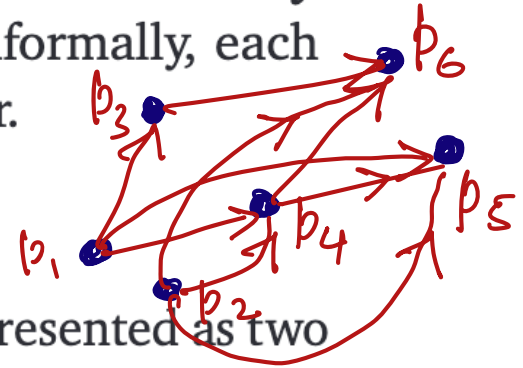


$$\sum \deg = 8n^2 = O(n^2)$$

$$O(|E| + |V| \log |V|) \approx O(n^2 + n^2 \log n)$$

Breadth First Search starting from (1,1).

A **polygonal path** is a sequence of line segments joined end-to-end; the endpoints of these line segments are called the **vertices** of the path. The **length** of a polygonal path is the sum of the lengths of its segments. A polygonal path with vertices $(x_1, y_1), (x_2, y_2), \dots, (x_k, y_k)$ is **monotonically increasing** if $x_i < x_{i+1}$ and $y_i < y_{i+1}$ for every index i —informally, each vertex of the path is above and to the right of its predecessor.



Suppose you are given a set S of n points in the plane, represented as two arrays $X[1..n]$ and $Y[1..n]$. Describe and analyze an algorithm to compute the length of the longest monotonically increasing path with vertices in S . Assume you have a subroutine $\text{LENGTH}(x, y, x', y')$ that returns the length of the segment from (x, y) to (x', y') .

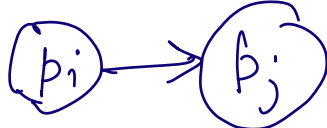
Graph Construction:

$V(G) = \{p_1, p_2, \dots, p_n\}$ set of all points

Coordinates are $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$



A directed edge from p_i to p_j
if $x_i < x_j$ and $y_i < y_j$.

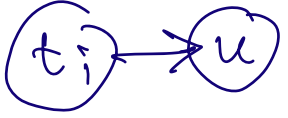
$\text{weight}(p_i, p_j)$ = the distance or length
between p_i and p_j . 

Preprocessing: Topological Sort of G .

Let $s_1, s_2, \dots, s_k$ be the vertices

s.t. there is no edge 

Let $t_1, t_2, t_3, \dots, t_r$ are the

vertices s.t. there is no edge 

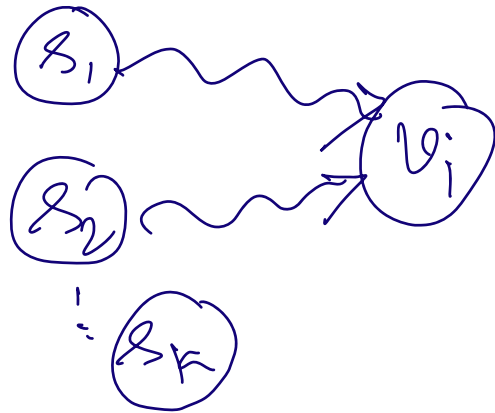
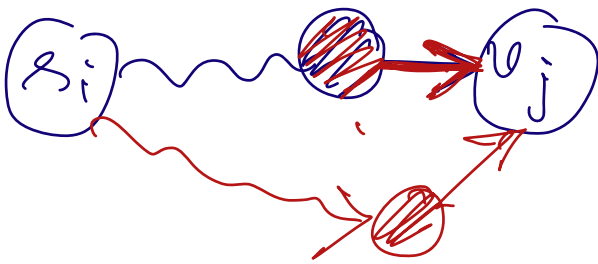
$\{v_1, v_2, \dots, v_n\}$ is the topological

order, $(v_1, \dots, v_k) = (\underline{s_1, \dots, s_k})$

$(v_{k+1}, \dots, v_n) = (t_1, \dots, t_r)$

Subproblem:

$LENGTH(s_i, j) =$ the length
of a longest path from
 s_i to v_j



Recurrence:

$$length(s_i, j) = \max_{\substack{u \in V(G): \\ (u, v_j) \text{ an edge}}} \left\{ \begin{array}{l} length(s_i, u) \\ + \text{weight}(u, v_j) \end{array} \right.$$

Base Case: $length(s_i, s_j) = 0$

for all $i, j \leq k$

Solution to the actual problem:

$$\max_{1 \leq j \leq n} \left\{ \text{Length}(s_i, t_j) \right\}$$

Algorithm:

not
final
answer

Initialization.
Storing the values in array
or table,
consistent with recurrence,
Output the